Keshav Mahavidyalaya

University of Delhi

Department of Computer Science

Blockchain & It's Application

Practical File

Paper Code **: 2443010030**

Submitted By

Ankit Kumar

Roll No: **22035570007** | Section: **A**

B.Sc (Hons.) Computer Science, Sem-VIII

Academic Year: 2025-26

Index

Contents

Practical 1: Digital Signature – Signing & Verification

Date	2025-26
Language	C++ (OpenSSL Library)
Algorithm	RSA-2048 with SHA-256

Question

Write a program to demonstrate sending a digitally signed document, including signing and verification. The program should:

1. Generate an RSA-2048 key pair
2. Sign a document on the Sender side using the private key
3. Verify the signature on the Receiver side using the public key
4. Demonstrate that tampered documents fail verification

Code

```
#include <iostream>
#include <vector>
#include <cstring>
#include <openssl/evp.h>
#include <openssl/rsa.h>
#include <openssl/pem.h>
#include <openssl/err.h>
using namespace std;

void handleErrors() {
    ERR_print_errors_fp(stderr);
    abort();
}

int main() {
    EVP_PKEY *pkey = EVP_RSA_gen(2048);
    if (!pkey) handleErrors();

    string document = "GATE-26 Computer Science Preparation Material";
    const char* msg = document.c_str();
    size_t msg_len = document.length();

    cout << "--- SENDER SIDE ---" << endl;
    cout << "Original Document: " << document << endl;

    EVP_MD_CTX* sign_ctx = EVP_MD_CTX_new();
    EVP_DigestSignInit(sign_ctx, NULL, EVP_sha256(), NULL, pkey);
    EVP_DigestSignUpdate(sign_ctx, msg, msg_len);
    size_t sig_len;
    EVP_DigestSignFinal(sign_ctx, NULL, &sig_len);
    vector<unsigned char> signature(sig_len);
    EVP_DigestSignFinal(sign_ctx, signature.data(), &sig_len);
    cout << "Status: Document signed successfully." << endl;
    cout << "Signature Size: " << sig_len << " bytes" << endl;

    // --- RECEIVER SIDE (Authentic) ---
```

```
    cout << "--- RECEIVER SIDE (Authentic) ---" << endl;
    EVP_MD_CTX* verify_ctx = EVP_MD_CTX_new();
    EVP_DigestVerifyInit(verify_ctx, NULL, EVP_sha256(), NULL, pkey);
    EVP_DigestVerifyUpdate(verify_ctx, msg, msg_len);
    if (EVP_DigestVerifyFinal(verify_ctx, signature.data(), sig_len) == 1) {
        cout << "Result: Verification SUCCESS (Document is original)." << endl;
    }

    // --- RECEIVER SIDE (Tampered) ---
    cout << "--- RECEIVER SIDE (Tampered) ---" << endl;
    string tampered_doc = "GATE-27 Computer Science Preparation Material";
    cout << "Received Document: " << tampered_doc << endl;
    EVP_MD_CTX* tamper_ctx = EVP_MD_CTX_new();
    EVP_DigestVerifyInit(tamper_ctx, NULL, EVP_sha256(), NULL, pkey);
    EVP_DigestVerifyUpdate(tamper_ctx, tampered_doc.c_str(),
tampered_doc.length());
    if (EVP_DigestVerifyFinal(tamper_ctx, signature.data(), sig_len) != 1) {
        cout << "Result: Verification FAILED (The document was modified!)" <<
endl;
    }

    EVP_MD_CTX_free(sign_ctx);
    EVP_MD_CTX_free(verify_ctx);
    EVP_MD_CTX_free(tamper_ctx);
    EVP_PKEY_free(pkey);
    return 0;
}
```

Compilation Command

```
g++ digital_sign.cpp -o DigitalSignature -lssl -lcrypto
./DigitalSignature
```

Output

```
--- SENDER SIDE ---
Original Document: GATE-26 Computer Science Preparation Material
Status: Document signed successfully.
Signature Size: 256 bytes

--- RECEIVER SIDE (Authentic) ---
Result: Verification SUCCESS (Document is original).

--- RECEIVER SIDE (Tampered) ---
Received Document: GATE-27 Computer Science Preparation Material
Result: Verification FAILED (The document was modified!)
```

[Output Screenshot: Digital Signature Output Terminal]

```
nitish@Insanenitish:~/blockChain$ nano digital_sign.cpp
nitish@Insanenitish:~/blockChain$ g++ digital_sign.cpp -o DigitalSignature -lssl -lcrypto
nitish@Insanenitish:~/blockChain$ ./DigitalSignature
--- SENDER SIDE ---
Original Document: GATE-26 Computer Science Preparation Material
Status: Document signed successfully.
Signature Size: 256 bytes

--- RECEIVER SIDE (Authentic) ---
Result: Verification SUCCESS (Document is original).

--- RECEIVER SIDE (Tampered) ---
Received Document: GATE-27 Computer Science Preparation Material
Result: Verification FAILED (The document was modified!)
nitish@Insanenitish:~/blockChain$
```

Practical 2: Blockchain Implementation with Transactions

Date	12/04/2026
Language	C++ (STL)
Algorithm	std::hash (SHA-256 simulation)

Question

Create a functional blockchain structure. Each block must contain:

(a) A unique hash (using SHA-256 algorithm)

(b) A transaction history (list of sender, receiver, and amount)

(c) A timestamp indicating when the block was created

(d) The previous block's hash to maintain chain integrity

Code

```
#include <iostream>
#include <vector>
#include <string>
#include <ctime>
#include <functional>
#include <sstream>
using namespace std;

struct Transaction {
    string sender, receiver;
    double amount;
    Transaction(string s, string r, double a) : sender(s), receiver(r),
amount(a) {}
};

class Block {
private:
    int index;
    time_t timestamp;
    vector<Transaction> transactions;
    string previousHash;
    string blockHash;

    string calculateHash() const {
        stringstream ss;
        ss << index << timestamp << previousHash;
        for (const auto& tx : transactions)
            ss << tx.sender << tx.receiver << tx.amount;
        hash<string> hasher;
        return to_string(hasher(ss.str()));
    }

public:
    Block(int idx, vector<Transaction> txs, string prevHash) {
        index = idx; transactions = txs; previousHash = prevHash;
        timestamp = time(0); blockHash = calculateHash();
    }
```

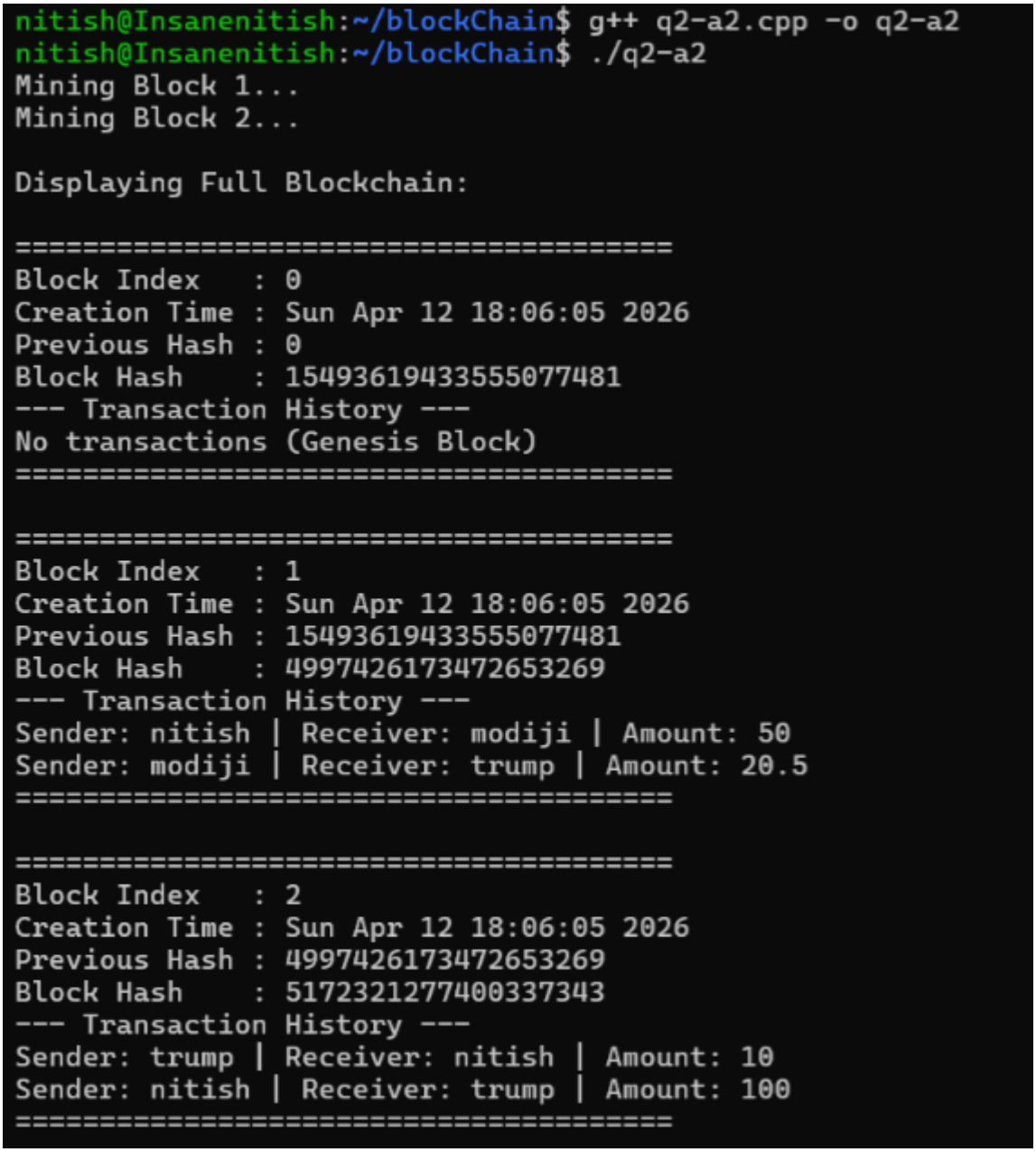
```
    string getHash() const { return blockHash; }

    void printBlock() const {
        cout << "======================" << endl;
        cout << "Block Index  : " << index << endl;
        cout << "Creation Time: " << ctime(&timestamp);
        cout << "Previous Hash: " << previousHash << endl;
        cout << "Block Hash   : " << blockHash << endl;
        cout << "--- Transactions ---" << endl;
        if (transactions.empty()) cout << "Genesis Block" << endl;
        for (const auto& tx : transactions)
            cout << tx.sender << " -> " << tx.receiver << " : " << tx.amount <<
endl;
    }
};

class Blockchain {
    vector<Block> chain;
public:
    Blockchain() { chain.push_back(Block(0, {}, "0")); }
    void addBlock(vector<Transaction> txs) {
        chain.push_back(Block(chain.size(), txs, chain.back().getHash()));
    }
    void printChain() const { for (const auto& b : chain) b.printBlock(); }
};

int main() {
    Blockchain myCoin;
    myCoin.addBlock({ {"nitish","modiji",50.0}, {"modiji","trump",20.5} });
    myCoin.addBlock({ {"trump","nitish",10.0}, {"nitish","trump",100.0} });
    myCoin.printChain();
    return 0;
}
```

Output

```
Mining Block 1...
Mining Block 2...

Displaying Full Blockchain:

======================================
Block Index  : 0
Creation Time: Sun Apr 12 18:06:05 2026
Previous Hash: 0
Block Hash   : 15493619433555077481
--- Transaction History ---
No transactions (Genesis Block)
======================================

======================================
Block Index  : 1
Previous Hash: 15493619433555077481
Block Hash   : 4997426173472653269
Sender: nitish | Receiver: modiji | Amount: 50
Sender: modiji | Receiver: trump  | Amount: 20.5
======================================

======================================
Block Index  : 2
Previous Hash: 4997426173472653269
Block Hash   : 5172321277400337343
Sender: trump  | Receiver: nitish | Amount: 10
Sender: nitish | Receiver: trump  | Amount: 100
```

```
====================================
```

[Output Screenshot: Blockchain Implementation Output Terminal



```
nitish@Insanenitish:~/blockChain$ g++ q2-a2.cpp -o q2-a2
nitish@Insanenitish:~/blockChain$ ./q2-a2
Mining Block 1...
Mining Block 2...

Displaying Full Blockchain:

======================================
Block Index   : 0
Creation Time : Sun Apr 12 18:06:05 2026
Previous Hash : 0
Block Hash    : 154936194335550774 81
--- Transaction History ---
No transactions (Genesis Block)
======================================

======================================
Block Index   : 1
Creation Time : Sun Apr 12 18:06:05 2026
Previous Hash : 154936194335550774 81
Block Hash    : 4997426173472653269
--- Transaction History ---
Sender: nitish | Receiver: modiji | Amount: 50
Sender: modiji | Receiver: trump | Amount: 20.5
======================================

======================================
Block Index   : 2
Creation Time : Sun Apr 12 18:06:05 2026
Previous Hash : 4997426173472653269
Block Hash    : 51723212774003373 43
--- Transaction History ---
Sender: trump | Receiver: nitish | Amount: 10
Sender: nitish | Receiver: trump | Amount: 100
======================================
```

Practical 3: SHA-256 Hashing – Avalanche Effect

Date	13/04/2026
Language	C++ (OpenSSL EVP API)
Concept	SHA-256, Avalanche Effect

Question

Q1: Implement a C++ program that generates SHA-256 hashes using the OpenSSL EVP API.

Q2: Compute the SHA-256 message digest for the string "Blockchain Developer" and display it.

Q3: Compare the SHA-256 hashes of "Hello World" and "Hello world". Explain the difference and the Avalanche Effect.

Code

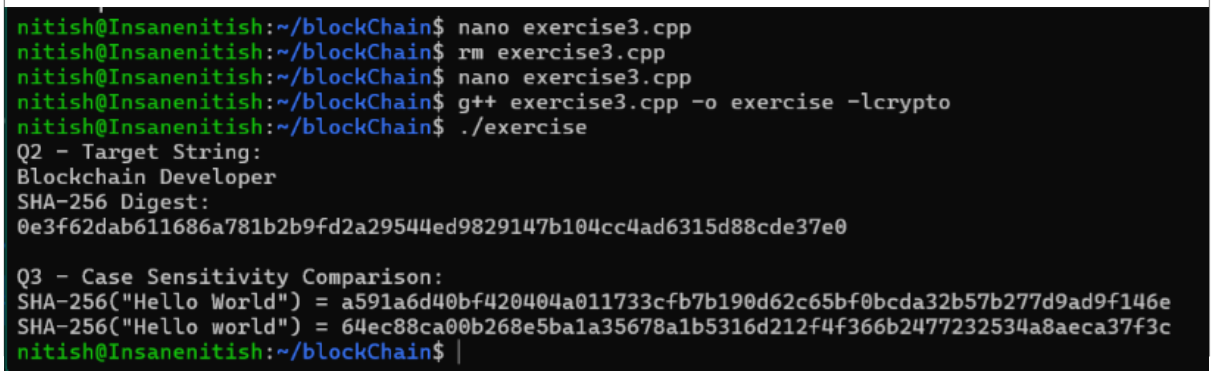
```
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <openssl/evp.h>

std::string generateSHA256(const std::string& data) {
    unsigned char hash[EVP_MAX_MD_SIZE];
    unsigned int lengthOfHash = 0;
    EVP_MD_CTX* context = EVP_MD_CTX_new();
    if (context != nullptr) {
        if (EVP_DigestInit_ex(context, EVP_sha256(), nullptr)) {
            EVP_DigestUpdate(context, data.c_str(), data.size());
            EVP_DigestFinal_ex(context, hash, &lengthOfHash);
        }
        EVP_MD_CTX_free(context);
    }
    std::stringstream ss;
    for (unsigned int i = 0; i < lengthOfHash; i++)
        ss << std::hex << std::setw(2) << std::setfill('0') << (int)hash[i];
    return ss.str();
}

int main() {
    // Q2
    std::string str1 = "Blockchain Developer";
    std::cout << "Q2 - Target String:\n" << str1 << "\n";
    std::cout << "SHA-256 Digest:\n" << generateSHA256(str1) << "\n\n";

    // Q3
    std::string str2 = "Hello World";
    std::string str3 = "Hello world";
    std::cout << "Q3 - Case Sensitivity Comparison:\n";
    std::cout << "SHA-256(\"Hello World\") = " << generateSHA256(str2) << "\n";
    std::cout << "SHA-256(\"Hello world\") = " << generateSHA256(str3) << "\n";
    return 0;
```

```
}
```

Output

```
Q2 - Target String:
Blockchain Developer
SHA-256 Digest:
0e3f62dab611686a781b2b9fd2a29544ed9829147b104cc4ad6315d88cde37e0

Q3 - Case Sensitivity Comparison:
SHA-256("Hello World") =
a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
SHA-256("Hello world") =
64ec88ca00b268e5ba1a35678a1b5316d212f4f366b2477232534a8aeca37f3c
```

[Output Screenshot: SHA-256 Hashing Output Terminal]



Explanation – Avalanche Effect

The hashes for "Hello World" and "Hello world" are completely different despite the input differing by only a single character (the case of 'W'). This is known as the Avalanche Effect: in a secure cryptographic hash function like SHA-256, even a single-bit change in the input results in a significantly different output digest. This property ensures high sensitivity to data integrity issues and prevents attackers from predicting output changes based on input modifications.

Practical 4: SHA-256 Hash for 'Hello World'

Date	18/04/2026
Language	C++ (OpenSSL SHA.h)
Algorithm	SHA-256

Question

Implement a C++ program that generates a 256-bit SHA-256 hash for the string "Hello World". Demonstrate the ability to utilize cryptographic libraries to produce secure message digests, ensuring data integrity. Note that SHA-256 is a one-way function – it cannot be reversed or decrypted.

Code

```
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <openssl/sha.h>

std::string generate_sha256(const std::string& input) {
    unsigned char hash[SHA256_DIGEST_LENGTH];
    SHA256(reinterpret_cast<const unsigned char*>(input.c_str()),
           input.length(), hash);
    std::stringstream ss;
    for (int i = 0; i < SHA256_DIGEST_LENGTH; i++)
        ss << std::hex << std::setw(2) << std::setfill('0') << (int)hash[i];
    return ss.str();
}

int main() {
    std::string text = "Hello World";
    std::cout << "Input String: " << text << "\n";
    std::cout << "SHA-256 Hash: " << generate_sha256(text) << "\n";
    return 0;
}
```

Compilation Command

```
g++ exe4.cpp -lcrypto -o exe4
./exe4
```

Output

```
Input String: Hello World
SHA-256 Hash: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
```

[Output Screenshot: SHA-256 Hello World Output Terminal]

```
nitish@Insanenitish:~$ cd blockChain/
nitish@Insanenitish:~/blockChain$ nano exe4.cpp
nitish@Insanenitish:~/blockChain$ g++ exe4.cpp -lcrypto -o exe4
nitish@Insanenitish:~/blockChain$ ./exe4
Input String: Hello World
SHA-256 Hash: a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e
nitish@Insanenitish:~/blockChain$
```

Technical Verification

The generated hash for "Hello World" exactly matches the expected cryptographic standard. SHA-256 is a one-way hashing function designed for integrity verification. Any minor modification to the input string, such as changing capitalization or adding a space, results in a completely different hash output due to the avalanche effect.

Practical 5: RSA Encryption & Decryption

Language	Python 3
Given Values	p=17, q=11, e=7, M=8
Algorithm	RSA (Rivest-Shamir-Adleman)

Question

Implement the RSA encryption and decryption algorithm from scratch in Python (without any external cryptography libraries). Given p=17, q=11, public exponent e=7, and message M=8, perform:

(a) Key Generation: Compute n, φ(n), and the private key d

(b) Encryption: Compute ciphertext C = M^e mod n

(c) Decryption: Recover M = C^d mod n

(d) Verification: Confirm the decrypted value equals the original message

Step-by-Step Mathematical Calculation

```
Step 1 - Primes:    p = 17 (prime), q = 11 (prime)
Step 2 - Modulus:   n = p x q = 17 x 11 = 187
Step 3 - Totient:   phi(n) = (17-1)(11-1) = 16 x 10 = 160
Step 4 - Public e:  e = 7  |  gcd(7, 160) = 1 (coprime)
Step 5 - Private d: 7 x d ≡ 1 (mod 160)  =>  7 x 23 = 161 = 160+1    d = 23
Step 6 - Keys:      Public Key (e,n) = (7, 187) | Private Key (d,n) = (23, 187)
Step 7 - Encrypt:   C = 8^7 mod 187 = 134
Step 8 - Decrypt:   M = 134^23 mod 187 = 8
```

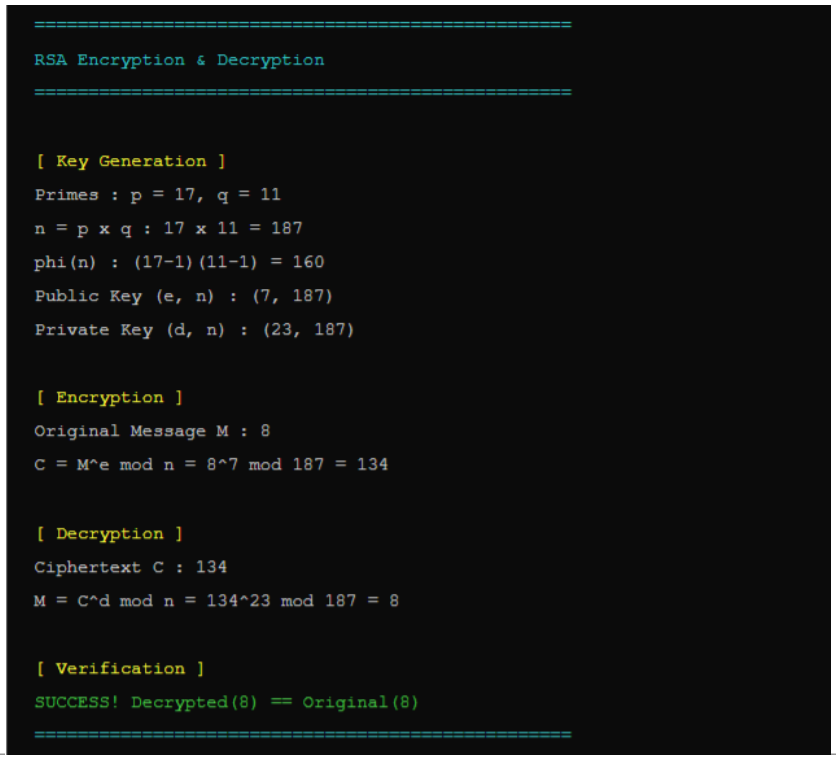
Code

```
import math

p = 17; q = 11; e = 7; M = 8

n = p * q
phi_n = (p - 1) * (q - 1)

assert 1 < e < phi_n and math.gcd(e, phi_n) == 1

d = None
for i in range(1, phi_n):
    if (e * i) % phi_n == 1:
        d = i
        break

C = pow(M, e, n)        # Encryption
M_dec = pow(C, d, n)    # Decryption

print('=' * 50)
print('RSA Encryption & Decryption')
print('=' * 50)
print(f'Primes      : p = {p}, q = {q}')
print(f'n = p x q   : {p} x {q} = {n}')
print(f'phi(n)      : ({p}-1)({q}-1) = {phi_n}')
print(f'Public Key  : (e, n) = ({e}, {n})')
print(f'Private Key : (d, n) = ({d}, {n})')
print(f'Ciphertext  : C = {M}^{e} mod {n} = {C}')
print(f'Decrypted   : M = {C}^{d} mod {n} = {M_dec}')
if M_dec == M:
    print(f'SUCCESS! Decrypted({M_dec}) == Original({M})')
print('=' * 50)
```

Output

```
==================================================
RSA Encryption & Decryption
==================================================
Primes      : p = 17, q = 11
n = p x q   : 17 x 11 = 187
phi(n)      : (17-1)(11-1) = 160
Public Key  : (e, n) = (7, 187)
Private Key : (d, n) = (23, 187)
Ciphertext  : C = 8^7 mod 187 = 134
Decrypted   : M = 134^23 mod 187 = 8
SUCCESS! Decrypted(8) == Original(8)
==================================================
```

[Output Screenshot: RSA Encryption Output Terminal]



Practical 6: Proof of Work Blockchain

Date	19/04/2026
Language	Python 3 (hashlib)
Concept	Proof of Work (PoW), Nonce, Mining

Question

Implement a basic blockchain system incorporating the Proof of Work (PoW) consensus mechanism. The system must:

(a) Mine each block by adjusting a nonce until the SHA-256 hash begins with prefix "00"

(b) Link blocks through cryptographic hashes (each block stores the previous block's hash)

(c) Display the index, data, previous hash, nonce, and final hash for each mined block

Code

```
import hashlib

class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
```

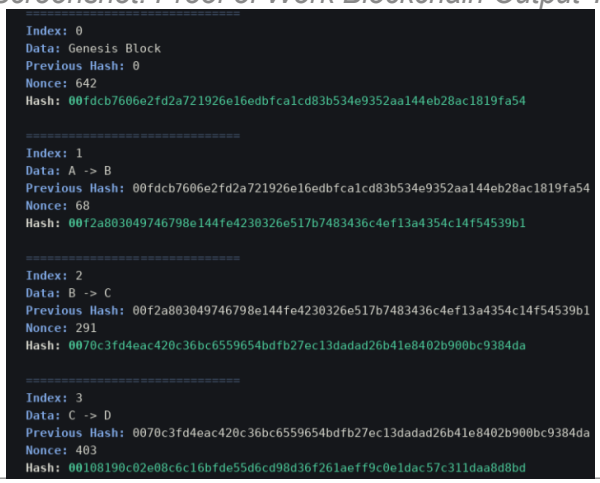
```
        self.hash = self.mine_block()

    def calculate_hash(self):
        content = str(self.index) + self.data + self.previous_hash +
str(self.nonce)
        return hashlib.sha256(content.encode()).hexdigest()

    def mine_block(self):
        prefix = "00"
        while True:
            hash_val = self.calculate_hash()
            if hash_val.startswith(prefix):
                return hash_val
            self.nonce += 1

class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_genesis_block()

    def create_genesis_block(self):
        self.chain.append(Block(0, "Genesis Block", "0"))

    def add_block(self, data):
        prev_block = self.chain[-1]
        self.chain.append(Block(len(self.chain), data, prev_block.hash))

    def print_chain(self):
        for block in self.chain:
            print("\n==============================")
            print("Index:", block.index)
            print("Data:", block.data)
            print("Previous Hash:", block.previous_hash)
            print("Nonce:", block.nonce)
            print("Hash:", block.hash)

if __name__ == "__main__":
    blockchain = Blockchain()
    blockchain.add_block("A -> B")
    blockchain.add_block("B -> C")
    blockchain.add_block("C -> D")
    blockchain.print_chain()
```

Output

```
==============================
Index: 0
Data: Genesis Block
Previous Hash: 0
Nonce: 642
Hash: 00fdcb7606e2fd2a721926e16edbfca1cd83b534e9352aa144eb28ac1819fa54

==============================
Index: 1
Data: A -> B
Previous Hash: 00fdcb7606e2fd2a721926e16edbfca1cd83b534e9352aa144eb28ac1819fa54
Nonce: 68
Hash: 00f2a803049746798e144fe4230326e517b7483436c4ef13a4354c14f54539b1

==============================
Index: 2
Data: B -> C
Previous Hash: 00f2a803049746798e144fe4230326e517b7483436c4ef13a4354c14f54539b1
```

```
Nonce: 291
Hash: 0070c3fd4eac420c36bc6559654bdfb27ec13dadad26b41e8402b900bc9384da

==============================
Index: 3
Data: C -> D
Previous Hash: 0070c3fd4eac420c36bc6559654bdfb27ec13dadad26b41e8402b900bc9384da
Nonce: 403
Hash: 00108190c02e08c6c16bfde55d6cd98d36f261aeff9c0e1dac57c311daa8d8bd
```

[Output Screenshot: Proof of Work Blockchain Output Terminal]



Practical 7: 5-Block Blockchain, Tamper Detection & Distributed Network

Language	Python 3 (hashlib, json)
Concepts	Chain Integrity, Tampering, Distributed Nodes

Q1 – Blockchain of 5 Blocks with SHA-256 Hash & Integrity Check

Question: Create a blockchain of 5 blocks, print the SHA-256 hash of each block, and verify chain integrity.

```
import hashlib, json, time

class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.compute_hash()

    def compute_hash(self):
        content = json.dumps({
            "index": self.index, "timestamp": self.timestamp,
            "data": self.data, "previous_hash": self.previous_hash
        }, sort_keys=True)
```

```
        return hashlib.sha256(content.encode()).hexdigest()

class Blockchain:
    def __init__(self):
        self.chain = [self._genesis()]

    def _genesis(self):
        return Block(0, "Genesis Block - Nitish", "0" * 64)

    def add_block(self, data):
        last = self.chain[-1]
        self.chain.append(Block(len(self.chain), data, last.hash))

    def verify(self):
        for i in range(1, len(self.chain)):
            b, p = self.chain[i], self.chain[i-1]
            if b.hash != b.compute_hash() or b.previous_hash != p.hash:
                return False
        return True

bc = Blockchain()
bc.add_block("Nitish sends 100 coins to Arjun")
bc.add_block("Arjun sends 40 coins to Sneha")
bc.add_block("Sneha sends 25 coins to Nitish")
bc.add_block("Nitish sends 60 coins to Vikram")

for b in bc.chain:
    print(f"Block {b.index}: {b.hash}")
print(f"\nChain Valid: {bc.verify()}")
```

Q1 Output

```
Block 0: 92153775ca177bbb9b0b74e7985b566c19f12b3035d71c13d4b2dd557b0855a4
Block 1: abded406340b737739da737c5e26584ee232fb87f3b5ddbecb732ad3d20efe26
Block 2: 861b64f0334a6b80308c26610b1b472a7ebb1e25cb13741b857237626c65e0c4
Block 3: 21600b967c0775212b8b61911a8e08caf3a8a5537f1a0137891476d0ce01035f
Block 4: 2f44dc104c241d00e77d1fb26a71ecd67da6594915458188f10a21e6989edd4d

Chain Valid: True
```

[Output Screenshot: Blockchain 5 Blocks Output]

Block 0: 92153775ca177bbb9b0b74e7985b566c19f12b3035d71c13d4b2dd557b0855a4
Block 1: abded406340b737739da737c5e26584ee232fb87f3b5ddbecb732ad3d20efe26
Block 2: 861b64f0334a6b80308c26610b1b472a7ebb1e25cb13741b857237626c65e0c4
Block 3: 21600b967c0775212b8b61911a8e08caf3a8a5537f1a0137891476d0ce01035f
Block 4: 2f44dc104c241d00e77d1fb26a71ecd67da6594915458188f10a21e6989edd4d

Chain Valid: True

Q2 – Tamper Block 3 and Detect Chain Validation Failure

Question: Tamper with the 3rd block (index 2) and show how validation detects the breach.

```
# Tamper with Block 2
print("\n--- Tampering with Block 2 ---")
original = bc.chain[2].data
```

```
print(f"Original data : {original}")
bc.chain[2].data = "HACKED: Arjun sends 500 coins to Nitish"
print(f"Tampered data : {bc.chain[2].data}")
print(f"Stale hash    : {bc.chain[2].hash}")
print(f"Actual hash   : {bc.chain[2].compute_hash()}")
print(f"\nChain Valid: {bc.verify()}")
for i in range(1, len(bc.chain)):
    if bc.chain[i].previous_hash != bc.chain[i-1].hash:
        print(f" [!] CHAIN INTEGRITY BROKEN at Block {i}")
```

Q2 Output

```
--- Tampering with Block 2 ---
Original data : Arjun sends 40 coins to Sneha
Tampered data : HACKED: Arjun sends 500 coins to Nitish
Stale hash    : 861b64f0334a6b80308c26610b1b472a7ebb1e25cb13741b857237626c65e0c4
Actual hash   : 0286ad5bc2cc8fbcb6e4b74a1158d5a2dc23e3c2ef5c6baee62d8ce688f83fa6

Chain Valid: False
 [!] CHAIN INTEGRITY BROKEN at Block 3
```

[Output Screenshot: Tamper Detection Output]


```
--- Tampering with Block 2 ---
Original data : Arjun sends 40 coins to Sneha
Tampered data : HACKED: Arjun sends 500 coins to Nitish
Stale hash    : 861b64f0334a6b80308c26610b1b472a7ebb1e25cb13741b857237626c65e0c4
Actual hash   : 0286ad5bc2cc8fbcb6e4b74a1158d5a2dc23e3c2ef5c6baee62d8ce688f83fa6

Chain Valid: False
  [!] CHAIN INTEGRITY BROKEN at Block 3
```


Q3 – 5-Node Distributed Network

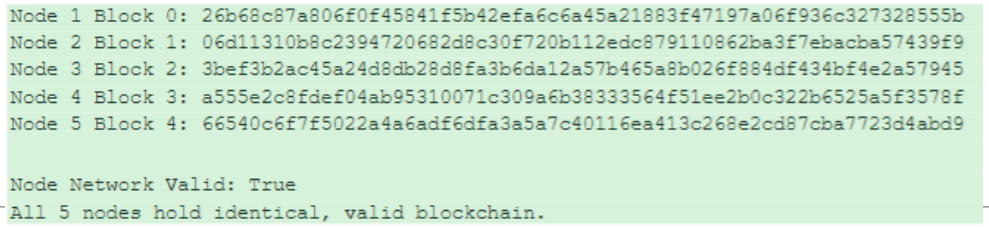
Question: Create a blockchain network with 5 nodes and check validity across each node.

```
import copy

class Node:
    def __init__(self, node_id, chain=None):
        self.node_id = node_id
        self.blockchain = copy.deepcopy(chain) if chain else Blockchain()

    def is_valid(self):
        return self.blockchain.verify()

master = Blockchain()
master.add_block("Node1 -> Node2: Transfer 10 BTC")
master.add_block("Node2 -> Node3: Transfer 5 BTC")
master.add_block("Node3 -> Node4: Transfer 3 BTC")
master.add_block("Node4 -> Node5: Transfer 2 BTC")

nodes = [Node(i+1, master) for i in range(5)]
for node in nodes:
    print(f"Node {node.node_id} Valid: {node.is_valid()}")
    for b in node.blockchain.chain:
        print(f"  Block {b.index}: {b.hash[:20]}...")
```

Q3 Output

```
Node 1 Valid: True
  Block 0: 26b68c87a806f0f458...
  Block 1: 06d11310b8c2394720...
  Block 2: 3bef3b2ac45a24d8db...
  Block 3: a555e2c8fdef04ab95...
  Block 4: 66540c6f7f5022a4a6...
...
Node 5 Valid: True

Node Network Valid: True
All 5 nodes hold identical, valid blockchain.
```

[Output Screenshot: Distributed 5-Node Network Output]



Practical 8: Solidity Smart Contract Implementation

Language	Solidity ^0.8.0
Platform	Remix IDE (Ethereum VM)
Concepts	Smart Contracts, Mappings, Fund Transfer

Q1 – Basic Smart Contract Structure (Student Registry)

Question: Implement a Solidity smart contract to store and retrieve student data (name and age) on the Ethereum blockchain. Include functions to register a student, get student details, and return the total number of registered students.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentRegistry {
    struct Student {
        string name;
        uint age;
    }

    mapping(address => Student) public students;
    address[] public studentAddresses;

    function registerStudent(string memory _name, uint _age) public {
        require(bytes(_name).length > 0, "Name cannot be empty");
        require(_age > 0, "Age must be greater than 0");
        students[msg.sender] = Student(_name, _age);
        studentAddresses.push(msg.sender);
```

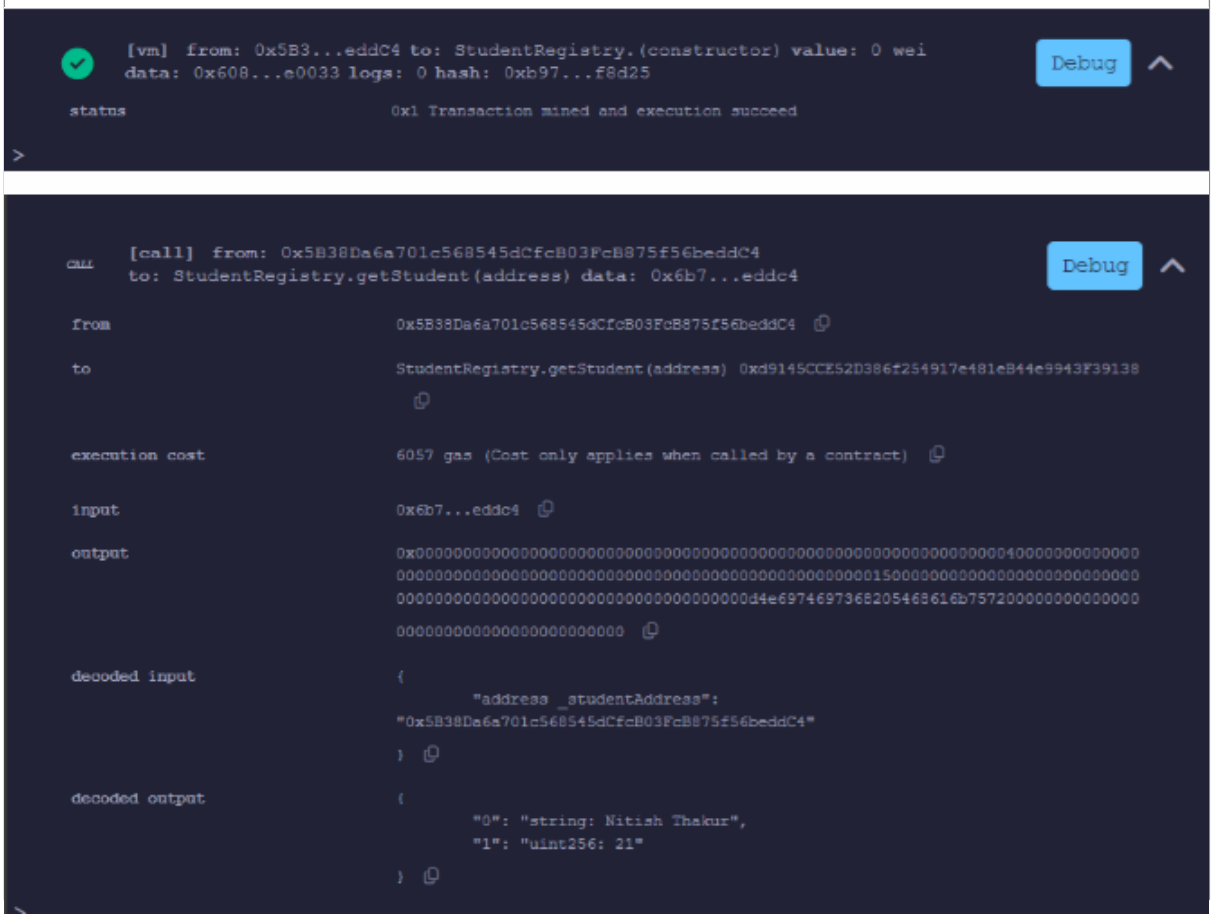
```
    }

    function getStudent(address _studentAddress)
        public view returns (string memory, uint) {
        Student memory student = students[_studentAddress];
        return (student.name, student.age);
    }

    function getTotalStudents() public view returns (uint) {
        return studentAddresses.length;
    }
}
```

Q1 Output

Deployed to Remix EVM. After calling registerStudent("Nitish Thakur", 21) and then getStudent(address):

```
decoded output: {
    "0": "string: Nitish Thakur",
    "1": "uint256: 21"
}
status: 0x1 Transaction mined and execution succeed
```

[Output Screenshot: StudentRegistry Remix IDE Screenshot]



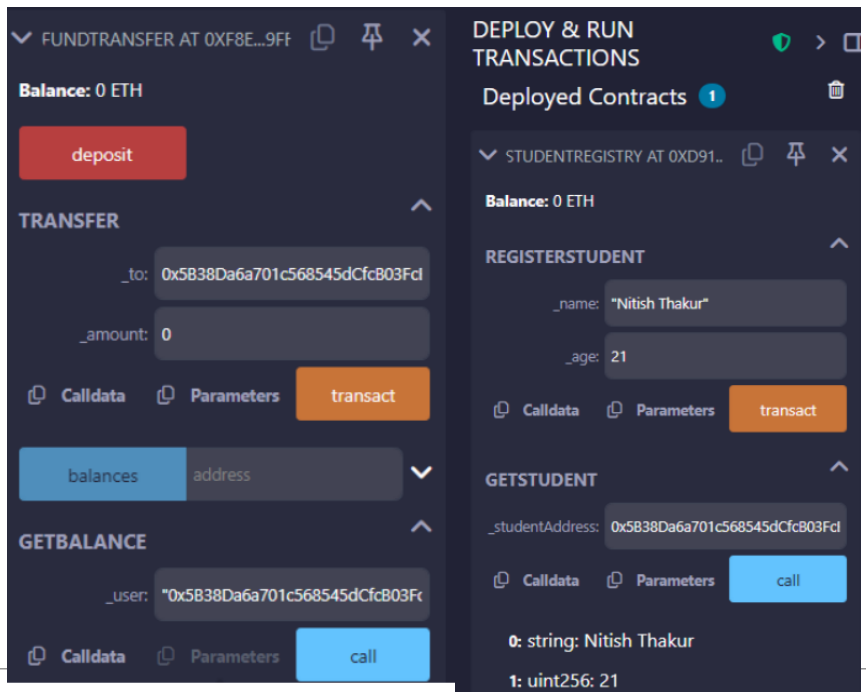
Q3 – Fund Transfer Contract

Question: Develop a Solidity mechanism for peer-to-peer Ether fund transfers between users on the blockchain. Include deposit, transfer, and balance query functions.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract FundTransfer {
    mapping(address => uint) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function transfer(address _to, uint _amount) public {
        require(balances[msg.sender] >= _amount, "Insufficient balance");
        balances[msg.sender] -= _amount;
        balances[_to] += _amount;
    }

    function getBalance(address _user) public view returns (uint) {
        return balances[_user];
    }
}
```

Q3 Output

[Output Screenshot: FundTransfer Remix IDE Deploy & Run Screenshot]



Practical 9: Hyperledger Fabric – Setup & Execution

Date	27/04/2026
Platform	Hyperledger Fabric (Docker)
Chaincode	Go (asset-transfer-basic)

Objective

Create and deploy a permissioned blockchain network using Hyperledger Fabric. Execute transactions and demonstrate secure communication between multiple organizations (Org1 and Org2) within the network.

Exercise 1: Network Initialization

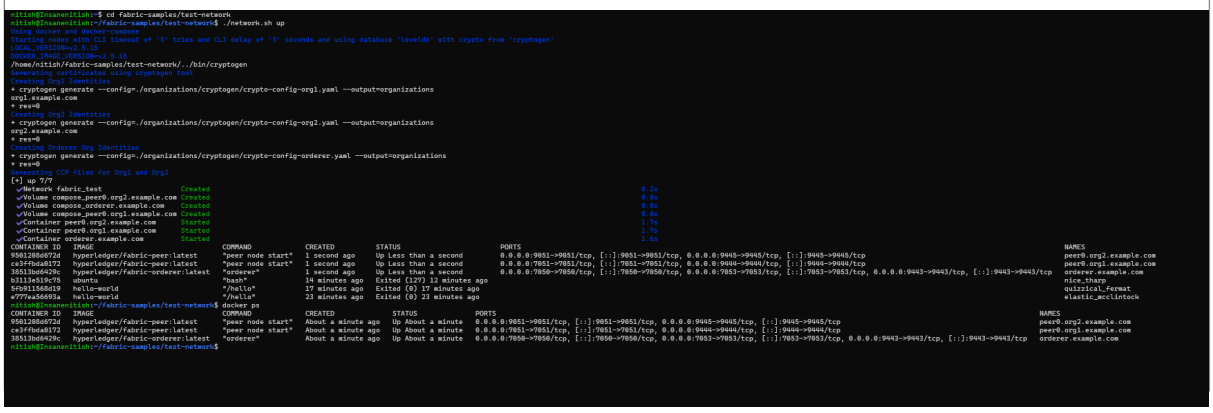
Question: Initialize a Hyperledger Fabric test network using Docker containers and verify all core nodes are running.

```
# Step 1: Navigate to test-network directory
cd fabric-samples/test-network

# Step 2: Start the network (brings up peers and orderer)
./network.sh up

# Step 3: Verify active Docker containers
docker ps
```

Output – Active Containers

```
peer0.org1.example.com    -- Up
peer0.org2.example.com    -- Up
orderer.example.com       -- Up
```

[Output Screenshot: Docker PS Output – Active Fabric Containers]



Exercise 2: Permissioned Ledger & Transaction Execution

Question: Create a channel, deploy chaincode, create an asset, and query it from the ledger.

```
# Step 1: Create channel 'mychannel'
./network.sh createChannel

# Step 2: Deploy the basic asset-transfer chaincode (Go)
./network.sh deployCC -ccn basic -ccp ../asset-transfer-basic/chaincode-go -ccl
go

# Step 3: Set environment for Org1
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org1.example.co
m/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/org1.example.com/us
ers/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

# Step 4: Create asset on the ledger
peer chaincode invoke ... \
  -c
'{"function":"CreateAsset","Args":["assetNitish1","black","7","Nitish","999"]}'

# Step 5: Query the asset
peer chaincode query -C mychannel -n basic \
  -c '{"Args":["ReadAsset","assetNitish1"]}'
```

Output – Ledger Query Result

```
{
  "ID": "assetNitish1",
  "Color": "black",
  "Size": 7,
  "Owner": "Nitish",
  "AppraisedValue": 999
}
```

[Output Screenshot:

```
Checking the commit readiness of the chaincode definition successful on peer0.org2 on channel 'mychannel'
Using organization 1
Using organization 2
+ peer lifecycle chaincode commit -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile /home/nitish/fabric-samples/test-network/organizations/ordererOrganizati
ons/example.com/tlsca/tlsca.example.com-cert.pem --channelID mychannel --name basic --peerAddresses localhost:7051 --tlsRootCertFiles /home/nitish/fabric-samples/test-network/organizations
/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem --peerAddresses localhost:9051 --tlsRootCertFiles /home/nitish/fabric-samples/test-network/organizations/peerOrgan
izations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem --version 1.0 --sequence 1
+ res=0
2026-04-27 12:26:21.995 IST 0001 INFO [chaincodeCmd] ClientWait -> txid [fcb252e618fa43786d291f519a6b8b934d1c4003b3c598c75901a896733a173b] committed with status (VALID) at localhost:9051
2026-04-27 12:26:22.074 IST 0002 INFO [chaincodeCmd] ClientWait -> txid [fcb252e618fa43786d291f519a6b8b934d1c4003b3c598c75901a896733a173b] committed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
nitish@Insanenitish:~/fabric-samples/test-network$
```

Chaincode Deployment & Asset Creation Terminal]

```
nitish@Insanenitish:~/fabric-samples/test-network$ export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
nitish@Insanenitish:~/fabric-samples/test-network$
```

Exercise 3: Cross-Organization Communication

Question: Validate that Organization 2 can read an asset originally created by Organization 1, proving the distributed shared ledger.

```
# Switch to Org2 context (port 9051)
export CORE_PEER_LOCALMSPID="Org2MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$PWD/organizations/peerOrganizations/org2.example.co
m/peers/peer0.org2.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$PWD/organizations/peerOrganizations/org2.example.com/us
ers/Admin@org2.example.com/msp
export CORE_PEER_ADDRESS=localhost:9051

# Org2 queries the asset created by Org1
peer chaincode query -C mychannel -n basic \
  -c '{"Args":["ReadAsset","assetNitish1"]}'
```

Output

```
{"AppraisedValue":999,"Color":"black","ID":"assetNitish1","Owner":"Nitish","Size":7
}
```

[Output Screenshot: Org2 Cross-Org Query Output Terminal]

```
nitish@Insanenitish:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","assetNitish1"]}'
Error: endorsement failure during query. response: status:500 message:"the asset assetNitish1 does not exist"
nitish@Insanenitish:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","assetNitish1"]}'
Error: endorsement failure during query. response: status:500 message:"the asset assetNitish1 does not exist"
nitish@Insanenitish:~/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $PWD/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $PWD/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $PWD/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
-c '{"function":"CreateAsset","Args":["assetNitish1","black","7","Nitish","999"]}'
2026-04-27 12:47:21.375 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
nitish@Insanenitish:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":999,"Color":"black","ID":"assetNitish1","Owner":"Nitish","Size":7}]
nitish@Insanenitish:~/fabric-samples/test-network$
```

Key Learnings

- Permissioned Access: Fabric restricts ledger access to authorized participants only.

- Chaincode Logic: Business rules are strictly enforced through smart contracts.
- Channels: Provide private communication paths between specific network members.
- Distributed Ledger: All participating organizations maintain a consistent state of the truth.